

熵编码详解

熵编码你可以把它理解成一句话：

根据符号出现概率来分配码长，常出现的符号用短码，少出现的符号用长码，从而在不丢信息的前提下减少平均比特数。

所以答案是：

Huffman 编码是熵编码。
算术编码也是熵编码。

它们都属于无损压缩中的统计编码，只是实现方式不同。

1. 熵编码是什么

熵编码的“熵”，指的是信息熵。

信息熵表示一个信源平均每个符号包含多少信息量，也可以理解成：

理论上平均每个符号最少需要多少 bit 来表示

如果一个符号很常见，它带来的“惊讶程度”低，信息量小。

如果一个符号很少见，它带来的“惊讶程度”高，信息量大。

所以熵编码的基本想法就是：

常见符号：信息量小 → 用短码
少见符号：信息量大 → 用长码

这样整体平均码长就会降低。

2. 信息量怎么算

某个符号 x 出现的概率是：

$p(x)$

它的信息量是：

$$I(x) = -\log_2 p(x)$$

也可以写成：

$$I(x) = \log_2(1 / p(x))$$

这个公式是什么意思

如果一个符号出现概率很大：

$p(x)$ 大
 $1 / p(x)$ 小
 $I(x)$ 小

说明它很常见，不需要很多 bit 表示。

如果一个符号出现概率很小：

$p(x)$ 小
 $1 / p(x)$ 大
 $I(x)$ 大

说明它很少见，需要更多 bit 表示。

比如：

$$p(A) = 1/2$$
$$I(A) = -\log_2(1/2) = 1 \text{ bit}$$

这表示如果 A 出现概率是 1/2，那么理论上 A 大约需要 1 bit 表示。

再比如：

$$p(B) = 1/8$$
$$I(B) = -\log_2(1/8) = 3 \text{ bit}$$

B 更少见，所以理论上需要更多 bit。

3. 信息熵怎么算

信息熵是所有符号信息量的加权平均。

公式是：

$$H(X) = \sum p(x_i) \cdot I(x_i)$$

因为：

$$I(x_i) = -\log_2 p(x_i)$$

所以也可以写成：

$$H(X) = - \sum p(x_i) \cdot \log_2 p(x_i)$$

这个公式是什么意思

信息熵不是看某一个符号，而是看整个信源平均每个符号的信息量。

也就是：

每个符号的信息量 × 这个符号出现的概率

全部加起来。

所以熵是一个理论下限：

平均码长不可能长期低于信息熵

如果某个编码的平均码长非常接近熵，说明它已经很接近最优压缩了。

4. 举一个完整例子

假设有 4 个符号：

符号	概率
A	1/2
B	1/4
C	1/8

符号	概率
D	1/8

分别计算信息量：

$$\begin{aligned}
 I(A) &= -\log_2(1/2) = 1 \\
 I(B) &= -\log_2(1/4) = 2 \\
 I(C) &= -\log_2(1/8) = 3 \\
 I(D) &= -\log_2(1/8) = 3
 \end{aligned}$$

再计算熵：

$$\begin{aligned}
 H(X) &= 1/2 \times 1 \\
 &\quad + 1/4 \times 2 \\
 &\quad + 1/8 \times 3 \\
 &\quad + 1/8 \times 3 \\
 &= 0.5 + 0.5 + 0.375 + 0.375 \\
 &= 1.75 \text{ bit/符号}
 \end{aligned}$$

意思是：

这个信源理论上平均每个符号至少需要约 **1.75 bit** 表示。

如果我们用等长编码，4 个符号都用 2 bit：

$$\begin{aligned}
 A &= 00 \\
 B &= 01 \\
 C &= 10 \\
 D &= 11
 \end{aligned}$$

平均码长就是：

$$2 \text{ bit/符号}$$

而熵是：

$$1.75 \text{ bit/符号}$$

说明等长编码还有冗余，还有压缩空间。

5. 平均码长怎么算

假设每个符号的码长是：

$$l(x_i)$$

平均码长是：

$$L = \sum p(x_i) \cdot l(x_i)$$

也就是：

每个符号的码长 $\times$ 出现概率

全部加起来。

判断一个编码好不好，主要看：

L 是否接近 $H(X)$

如果：

$$L \gg H(X)$$

说明编码冗余很多。

如果：

$$L \approx H(X)$$

说明编码效率很高。

6. Huffman 为什么是熵编码

Huffman 编码的核心规则是：

概率大的符号 $\rightarrow$ 短码

概率小的符号 $\rightarrow$ 长码

这正是熵编码的思想。

Huffman 编码怎么做

步骤如下：

1. 统计每个符号出现的概率
2. 每次选出概率最小的两个符号或节点
3. 把它们合并成一个新节点，概率相加
4. 重复合并，直到形成一棵树
5. 从树根到叶子分配 0/1，得到每个符号的码字

每一步的意图

第一步，统计概率：

知道谁常见，谁少见

第二步，合并最小概率：

让最少见的符号处在树的更深处

树越深，码字越长。

第三步，逐步形成树：

自动生成一种前缀码

前缀码的意思是：

任何一个码字都不是另一个码字的前缀

这样解码时不会混淆。

比如如果有：

A = 0
B = 01

那看到 01 时就不知道是 A 后面还有东西，还是 B。

Huffman 编码会避免这种问题。

Huffman 的局限

Huffman 的码长必须是整数 bit。

比如一个符号理论上最好用：

1.3 bit

但 Huffman 不可能真的给它 1.3 bit，只能给 1 bit 或 2 bit。

所以 Huffman 的平均码长通常满足：

$$L > H(X)$$

也就是接近熵，但一般不能完全达到熵。

7. 算术编码为什么也是熵编码

算术编码也是根据符号概率来压缩，只是它不是：

一个符号 → 一个码字

而是：

一整串符号 → 一个 0 到 1 之间的小数

所以它也是熵编码。

算术编码的基本思想

假设整个区间是：

$[0, 1)$

然后按照符号概率把这个区间切开。

例如：

符号	概率	区间
A	0.5	$[0, 0.5)$

符号	概率	区间
B	0.25	[0.5, 0.75)
C	0.25	[0.75, 1)

如果要编码的第一个符号是 A，就把当前区间缩小到：

```
[0, 0.5)
```

如果第二个符号是 C，就在 `[0, 0.5)` 里面再按 A/B/C 的概率切分，然后选择 C 对应的小区间。

不断重复，消息越长，区间越小。

最后只要在最终区间里任选一个小数，就可以代表整串消息。

算术编码怎么算

假设当前区间是：

```
[low, high)
```

区间长度是：

```
range = high - low
```

某个符号在概率表中的累计区间是：

```
[cum_low, cum_high)
```

那么更新后的新区间是：

```
new_low  = low + range × cum_low  
new_high = low + range × cum_high
```

每读入一个符号，就更新一次区间。

每一步的意图

第一步，把 `[0, 1)` 按概率分区：

概率大的符号分到更宽的区间

概率小的符号分到更窄的区间

第二步，根据当前符号缩小区间：

把当前消息的信息逐步编码进区间位置

第三步，最终选一个小数：

用一个数表示整串消息

第四步，转换成二进制：

区间越宽，需要的二进制位越少

区间越窄，需要的二进制位越多

而高概率消息对应的最终区间通常更宽，所以需要更少 bit。

这仍然符合熵编码思想。

8. Huffman 和算术编码的区别

对比点	Huffman 编码	算术编码
基本单位	一个符号一个码字	一整串符号一个小数
码长	必须是整数 bit	平均意义上可以接近小数 bit
接近熵的能力	接近熵，但不一定很贴近	通常更接近熵
实现复杂度	较低	较高
典型问题	要保存码表，码长受整数限制	区间计算复杂，需要处理精度问题

所以可以这样理解：

Huffman：把概率变成一棵树

算术编码：把概率变成一个不断缩小的区间

9. 熵编码在 JPEG 里做什么

JPEG 里前面的步骤会让数据变得更适合熵编码：

DCT: 把能量集中到低频

量化: 让很多高频系数变成 0

Zig-Zag: 让 0 连续出现

RLE: 把连续 0 变成更短的符号

DPCM: 把 DC 系数变成较小的差值

熵编码: 对这些符号做最后的无损压缩

JPEG 常用 Huffman 编码, 也可以使用算术编码。

注意:

量化是有损的。

熵编码是无损的。

熵编码不会再丢图像信息, 它只是把已经得到的符号换成更短的二进制表示。

10. 每一步的总意图

步骤	做什么	意图
统计概率	统计每个符号出现频率	知道哪些符号常见
计算信息量	$I(x) = -\log_2 p(x)$	衡量单个符号含有多少信息
计算熵	$H(X) = -\sum p \log_2 p$	得到理论平均码长下限
设计码字	常见短码, 少见长码	降低平均码长
计算平均码长	$L = \sum p \cdot l$	判断实际编码效果
比较 L 和 H	看是否接近熵	判断编码效率

考试背诵版

可以这样背:

熵编码是一类基于信源概率分布的无损压缩编码方法。它利用不同符号出现概率不同的特点, 为高概率符号分配短码, 为低概率符号分配长码, 从而降低平均码长。信息量公式为 $I(x) = -\log_2 p(x)$, 信息熵公式为 $H(X) = -\sum p(x) \log_2 p(x)$, 熵表示平均每个符号所需比特数的理论下限。Huffman 编码和算术编码都属于熵编码。Huffman 通过构造二叉树生成前缀码, 简单高效, 但码长必须是整数, 平均码长一般只能接近熵; 算术编码把整个消息表示为

$[0,1)$ 区间中的一个小数，平均码长可以更接近熵极限。熵编码本身不丢失信息，在 JPEG 中通常位于量化、Zig-Zag、RLE 和 DPCM 之后，用于对中间符号进行进一步无损压缩。

最关键的一句：

熵编码不是负责“丢信息”，而是负责“用更短的码表示更常见的符号”。

Huffman 和算术编码都是熵编码。